

Instituto Tecnológico de Villahermosa

Tecnológico Nacional de México

Materia:

Aplicaciones Móviles

Nombre del alumno:

Juan de Dios Díaz Gómez

Número de control:

22050566

Docente:

Jonathan de la Cruz Alvarez

Carrera:

Ingeniería en Sistemas Computacionales

Grupo:

DAB2306_A

Actividad:

U4-T2 Aplicación Final

Fecha:

25 de Mayo de 2026

Introducción

El presente documento constituye la evidencia técnica y académica correspondiente a la actividad **U4-T2 Aplicación Final**, desarrollada en el marco de la materia *Aplicaciones Móviles* de la carrera de Ingeniería en Sistemas Computacionales del Instituto Tecnológico de Villahermosa. El proyecto tiene como objetivo demostrar la integración completa de un sistema de información compuesto por una API RESTful desarrollada con el framework **Yii 2** (PHP) y una aplicación móvil multiplataforma construida con **ionic 8 + Angular 20**, empaquetada con Capacitor para su despliegue en dispositivos Android e iOS.

La aplicación, denominada **Minecraft: Panel de Control**, permite gestionar de forma integral los recursos de un servidor de Minecraft: mundos, biomas, jugadores, mobs, items e inventarios. Cada módulo expone operaciones CRUD completas a través de endpoints RESTful que son consumidos por la interfaz móvil mediante peticiones HTTP con autenticación vía Bearer Token. La arquitectura implementada sigue el patrón MVC que impone Yii 2 en el lado del servidor, mientras que en el cliente se aplica la arquitectura basada en componentes y servicios de Angular.

Este documento incluye capturas del sistema en funcionamiento, descripción detallada de cada módulo, esquema de las tablas de base de datos asignadas, análisis del funcionamiento de la API RESTful con sus endpoints y verbos HTTP, así como reflexiones sobre los aprendizajes obtenidos durante el desarrollo de esta aplicación final.

Explicación General del Proyecto

Arquitectura del Sistema

El sistema se estructura en dos capas principales que se comunican mediante una API RESTful:

Capa	Tecnología	Función
Backend / API	Yii 2 Framework (PHP 8.2)	Expone endpoints REST, gestiona BD MySQL, autenticación Bearer Token
Frontend Móvil	Ionic 8 + Angular 20 + Capacitor	Interfaz de usuario móvil multiplataforma (Android/iOS/Web)
Base de Datos	MySQL — minecraftdb	Almacena todas las entidades del servidor Minecraft
Autenticación	HTTP Bearer Token	Token generado al login/registro; protege endpoints sensibles

Módulos de la Aplicación

La aplicación gestiona seis entidades principales del dominio de Minecraft, cada una con pantallas de listado, detalle, creación y edición, más dos módulos transversales de autenticación y permisos:

- **Mundos:** Dimensiones generadas en el servidor (nombre, semilla, dificultad).
- **Biomás:** Zonas geográficas con temperatura asociadas a un mundo.
- **Jugadores:** Usuarios registrados con nivel XP, rol y fecha de unión.
- **Mobs:** Criaturas del juego con tipo y nivel de hostilidad, vinculadas a un bioma.
- **Items:** Objetos del juego con atributos de apilabilidad y puntos de ataque.
- **Inventarios:** Relación entre jugadores e items con sus cantidades.
- **Autenticación:** Login, registro y gestión de tokens JWT-compatible.
- **Permisos:** Control de acceso por roles (jugador / administrador).

API RESTful — Subdominio y Funcionamiento

Subdominio y URL Base

La API RESTful corre localmente sobre el servidor de desarrollo integrado de PHP en el puerto **8080**. La URL base del servicio es:

```
http://localhost:8080/
```

En entorno de producción el backend se despliega bajo el subdominio del servidor asignado por la institución. Los endpoints siguen la convención RESTful estándar establecida por Yii 2 a través del componente yii\rest\UrlRule y yii\rest\ActiveController:

Recurso	Endpoint	Verbos HTTP soportados	Autenticación
Mundos	GET/POST /mundos	GET, POST, PUT, DELETE	Bearer Token
Biomás	GET/POST /biomas	GET, POST, PUT, DELETE	Bearer Token
Jugadores	GET/POST /jugadores	GET, POST, PUT, DELETE	Público (login/reg.)
Mobs	GET/POST /mobs	GET, POST, PUT, DELETE	Bearer Token
Items	GET/POST /items	GET, POST, PUT, DELETE	Bearer Token
Inventarios	GET/POST /inventarios	GET, POST, PUT, DELETE	Bearer Token
Permisos	GET /permiso	GET	Bearer Token
Jug.-Mundos	GET/POST /jugadores-mundos	GET, POST, PUT, DELETE	Bearer Token

Capturas de la API en Funcionamiento

A continuación se muestran capturas directas del navegador accediendo a los endpoints de la API en tiempo real, evidenciando el formato JSON de respuesta que consume la aplicación móvil:

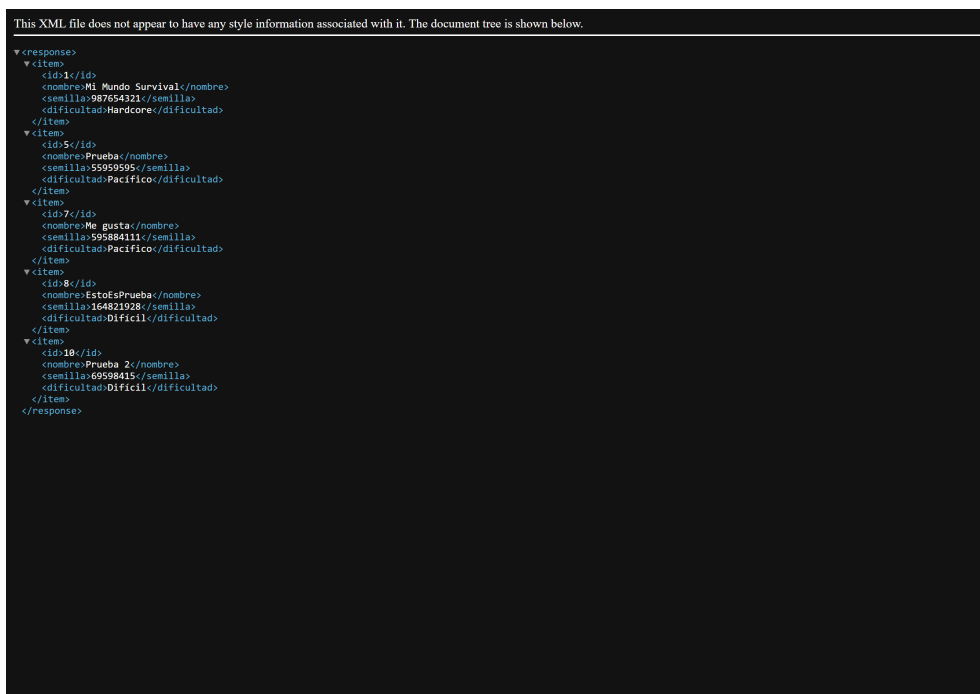


Figura 1 — GET /mundos: listado completo de mundos en formato JSON.

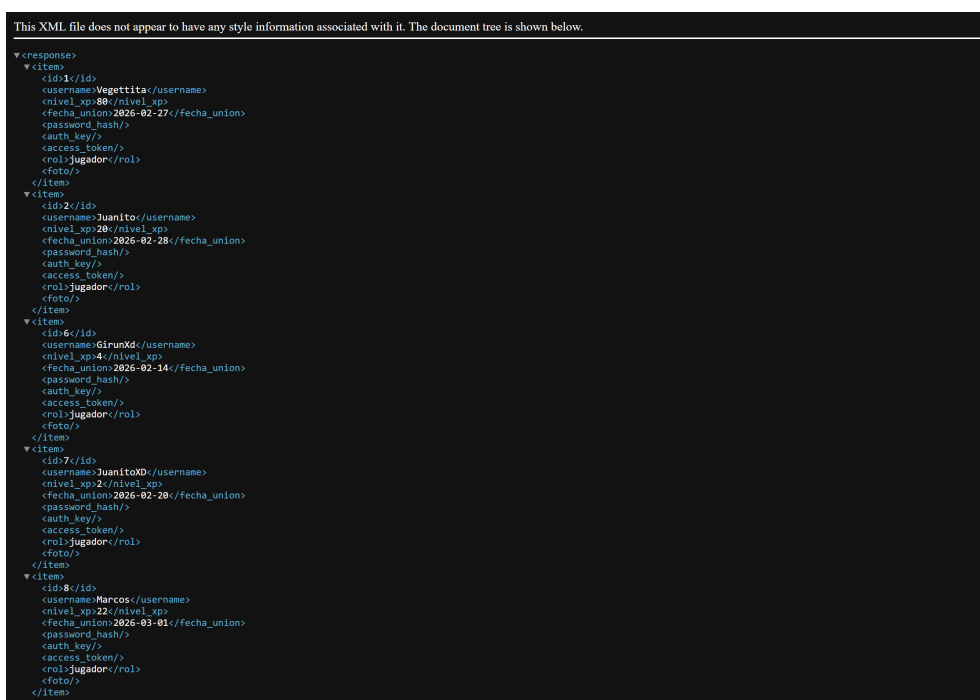


Figura 2 — GET /jugadores: registro de jugadores con token de acceso.

```
This XML file does not appear to have any style information associated with it. The document tree is shown below.

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<response>
  <item>
    <id>2</id>
    <nombre>Desierto de Almas</nombre>
    <temperatura>2.00</temperatura>
    <id_mundo>1</id_mundo>
  </item>
  <item>
    <id>3</id>
    <nombre>Me pela la verga</nombre>
    <temperatura>2.00</temperatura>
    <id_mundo>7</id_mundo>
  </item>
  <item>
    <id>4</id>
    <nombre>Pichon</nombre>
    <temperatura>-0.50</temperatura>
    <id_mundo>5</id_mundo>
  </item>
  <item>
    <id>8</id>
    <nombre>Una prueba</nombre>
    <temperatura>2.00</temperatura>
    <id_mundo>1</id_mundo>
  </item>
  <item>
    <id>9</id>
    <nombre>prueba una vez</nombre>
    <temperatura>2.00</temperatura>
    <id_mundo>7</id_mundo>
  </item>
  <item>
    <id>10</id>
    <nombre>Chi, funciona</nombre>
    <temperatura>2.00</temperatura>
    <id_mundo>8</id_mundo>
  </item>
  <item>
    <id>11</id>
    <nombre>Prueba de la prueba</nombre>
    <temperatura>0.40</temperatura>
    <id_mundo>10</id_mundo>
  </item>
</response>
```

Figura 3 — GET /biomas: biomas con temperatura y mundo asociado.

```
This XML file does not appear to have any style information associated with it. The document tree is shown below.

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<response>
  <item>
    <id>1</id>
    <tipo>Te pilles</tipo>
    <es_hostil>1</es_hostil>
    <id_bioma>2</id_bioma>
  </item>
  <item>
    <id>2</id>
    <tipo>Pokemon</tipo>
    <es_hostil>0</es_hostil>
    <id_bioma>3</id_bioma>
  </item>
  <item>
    <id>3</id>
    <tipo>El RubiusOMG</tipo>
    <es_hostil>0</es_hostil>
    <id_bioma>4</id_bioma>
  </item>
  <item>
    <id>4</id>
    <tipo>Sexo oral</tipo>
    <es_hostil>0</es_hostil>
    <id_bioma>2</id_bioma>
  </item>
  <item>
    <id>8</id>
    <tipo>Esto es prueba</tipo>
    <es_hostil>0</es_hostil>
    <id_bioma>8</id_bioma>
  </item>
  <item>
    <id>9</id>
    <tipo>veamos</tipo>
    <es_hostil>1</es_hostil>
    <id_bioma>2</id_bioma>
  </item>
</response>
```

Figura 4 — GET /mobs: criaturas con tipo y bioma vinculado.

API RESTful — Items, Inventarios y Permisos

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<response>
  <item>
    <id>1</id>
    <nombre>Pico de Netherite</nombre>
    <es_apilable>0</es_apilable>
    <puntos_ataque>98</puntos_ataque>
  </item>
  <item>
    <id>5</id>
    <nombre>Pene</nombre>
    <es_apilable>0</es_apilable>
    <puntos_ataque>100</puntos_ataque>
  </item>
  <item>
    <id>7</id>
    <nombre>Espada de penezote</nombre>
    <es_apilable>0</es_apilable>
    <puntos_ataque>12</puntos_ataque>
  </item>
  <item>
    <id>8</id>
    <nombre>Pico de diamante</nombre>
    <es_apilable>0</es_apilable>
    <puntos_ataque>3</puntos_ataque>
  </item>
  <item>
    <id>9</id>
    <nombre>Espada de diamante</nombre>
    <es_apilable>1</es_apilable>
    <puntos_ataque>13</puntos_ataque>
  </item>
</response>
```

Figura 5 — GET /items: objetos del juego con atributos de combate.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<response>
  <item>
    <id>1</id>
    <id_jugador>1</id_jugador>
    <id_item>1</id_item>
    <cantidad>6</cantidad>
  </item>
  <item>
    <id>2</id>
    <id_jugador>1</id_jugador>
    <id_item>1</id_item>
    <cantidad>11</cantidad>
  </item>
  <item>
    <id>3</id>
    <id_jugador>7</id_jugador>
    <id_item>1</id_item>
    <cantidad>7</cantidad>
  </item>
  <item>
    <id>6</id>
    <id_jugador>7</id_jugador>
    <id_item>7</id_item>
    <cantidad>12</cantidad>
  </item>
  <item>
    <id>7</id>
    <id_jugador>8</id_jugador>
    <id_item>5</id_item>
    <cantidad>12</cantidad>
  </item>
  <item>
    <id>9</id>
    <id_jugador>6</id_jugador>
    <id_item>5</id_item>
    <cantidad>13</cantidad>
  </item>
  <item>
    <id>10</id>
    <id_jugador>2</id_jugador>
    <id_item>7</id_item>
    <cantidad>12</cantidad>
  </item>
  <item>
    <id>11</id>
    <id_jugador>7</id_jugador>
    <id_item>9</id_item>
    <cantidad>27</cantidad>
  </item>
  <item>
    <id>12</id>
    <id_jugador>1</id_jugador>
    <id_item>5</id_item>
    <cantidad>45</cantidad>
  </item>
</response>
```

Figura 6 — GET /inventarios: relación jugador–item con cantidades.

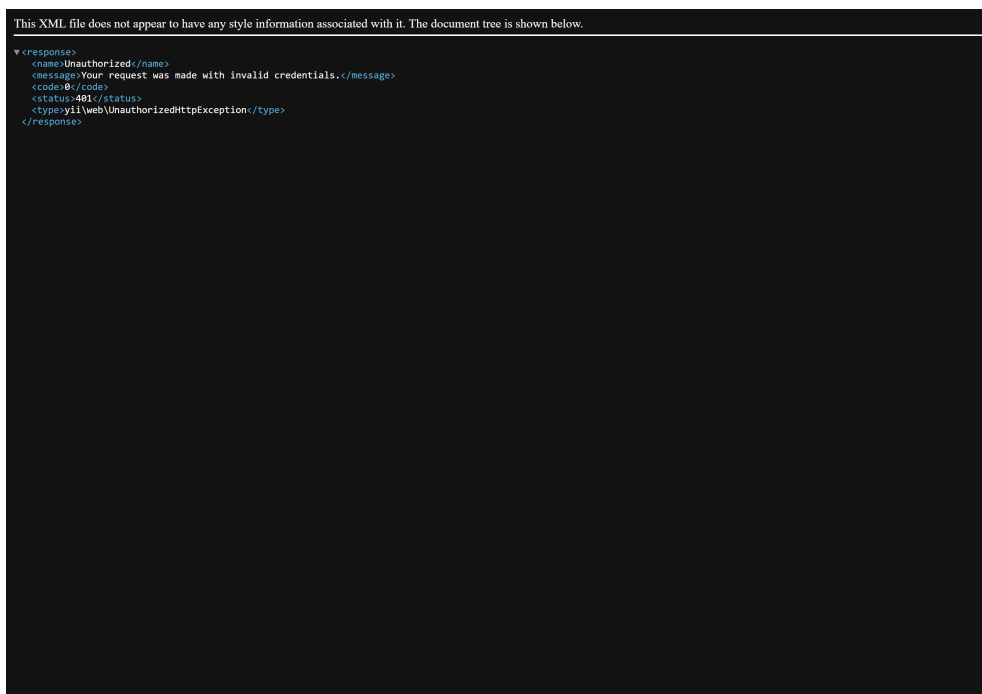


Figura 7 — GET /permiso: tabla de control de acceso por roles.

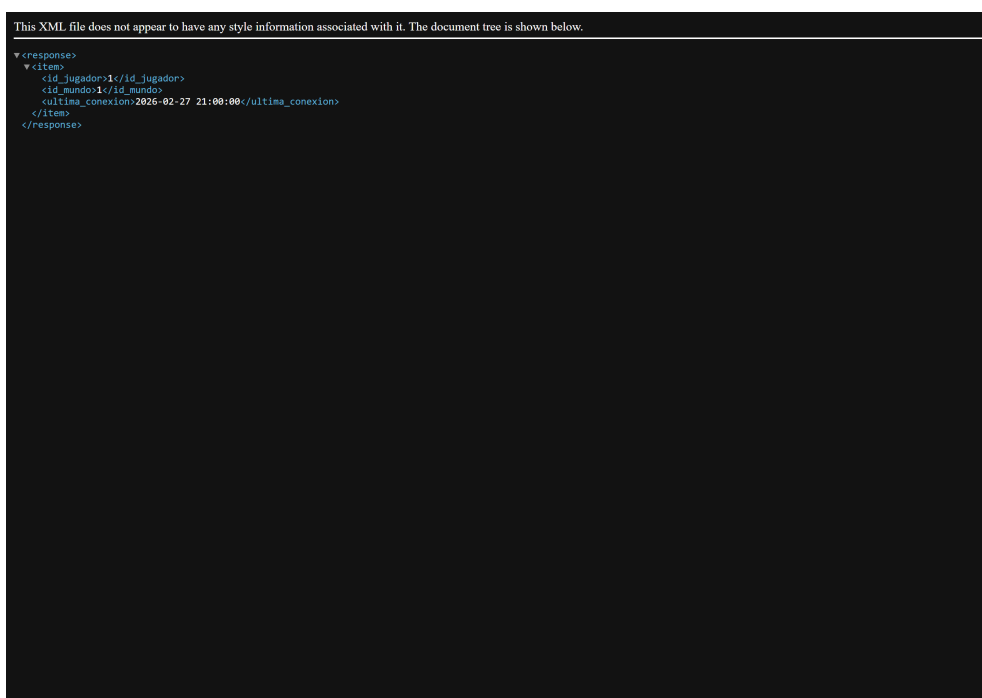


Figura 8 — GET /jugadores-mundos: relación muchos a muchos con timestamp.

Estructura de la Base de Datos — Tablas Asignadas

La base de datos **minecraftdb** contiene 9 tablas que modelan el dominio completo del servidor de Minecraft. A continuación se detalla la estructura de cada una:

Tabla: mundos

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
nombre	VARCHAR(50)	SÍ		—
semilla	BIGINT	SÍ		—
dificultad	VARCHAR(20)	SÍ		—

Tabla: biomas

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
nombre	VARCHAR(50)	SÍ		—
temperatura	DECIMAL(3,2)	SÍ		—
id_mundo	INT (FK)	SÍ	MUL	—

Tabla: mobs

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
tipo	VARCHAR(50)	SÍ		—
es_hostil	TINYINT(1)	SÍ		—
id_bioma	INT (FK)	SÍ	MUL	—

Tabla: jugadores

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
username	VARCHAR(50)	NO		—
nivel_xp	INT	SÍ		0
fecha_union	DATE	SÍ		—
password_hash	VARCHAR(255)	NO		—
auth_key	VARCHAR(32)	NO		—
access_token	VARCHAR(255)	SÍ		—

rol	VARCHAR(50)	NO		jugador
foto	VARCHAR(255)	SÍ		—

Tabla: items

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
nombre	VARCHAR(50)	SÍ		—
es_apilable	TINYINT(1)	SÍ		1
puntos_ataque	INT	SÍ		0

Tabla: inventarios

Campo	Tipo	Nulo	Clave	Default
id	INT (PK, AI)	NO	PRI	—
id_jugador	INT (FK)	SÍ	MUL	—
id_item	INT (FK)	SÍ	MUL	—
cantidad	INT	SÍ		—

Tabla: jugadores_mundos (muchos a muchos)

Campo	Tipo	Nulo	Clave	Default
id_jugador	INT (FK, PK)	NO	PRI	—
id_mundo	INT (FK, PK)	NO	PRI	—
ultima_conexion	DATETIME	SÍ		—

Tabla: permiso

Campo	Tipo	Nulo	Clave	Default
per_id	INT (PK, AI)	NO	PRI	—
per_vista	VARCHAR(100)	NO		—
per_rol	VARCHAR(150)	NO		—

Pantallas Finales de la Aplicación Móvil

Módulo de Autenticación

La aplicación comienza con un flujo de autenticación que incluye inicio de sesión y registro. El formulario de login valida credenciales contra la API y almacena el Bearer Token localmente para autenticar todas las peticiones subsiguientes. El registro incluye validación de contraseña con requisitos de complejidad (mín. 8 chars, mayúsculas, número y carácter especial).

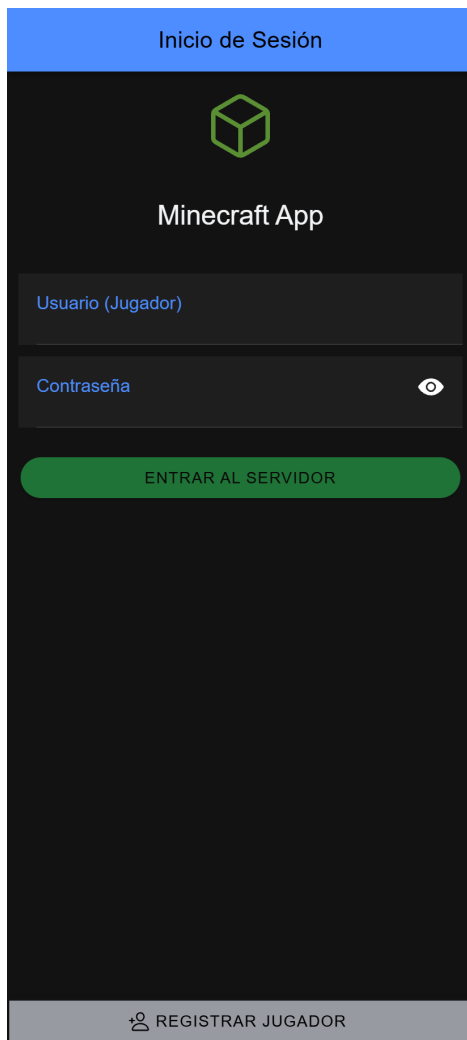


Figura 9 — Pantalla de inicio de sesión.

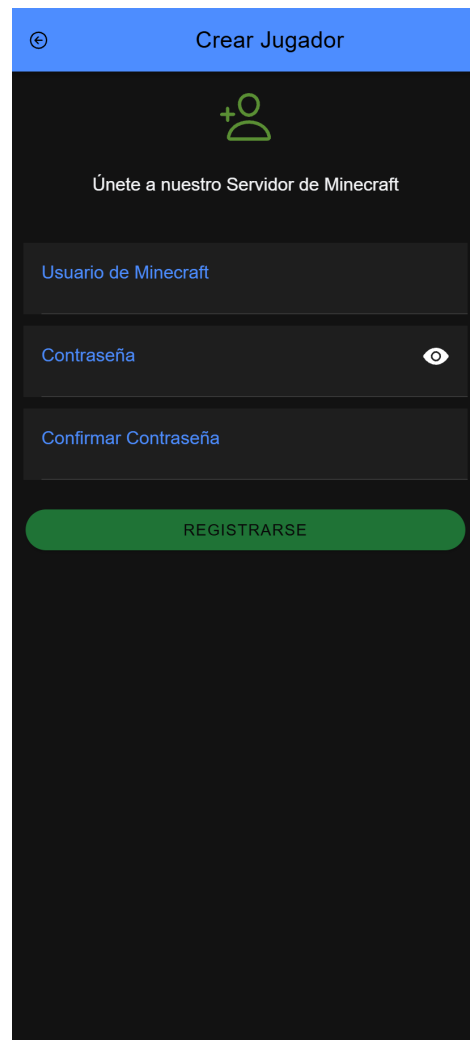


Figura 10 — Formulario de registro de nuevo jugador.

Panel de Bienvenida

Tras autenticarse, el usuario accede al Panel de Control donde se muestran los seis módulos principales mediante tarjetas interactivas sobre un fondo oscuro que recrea la estética del juego.

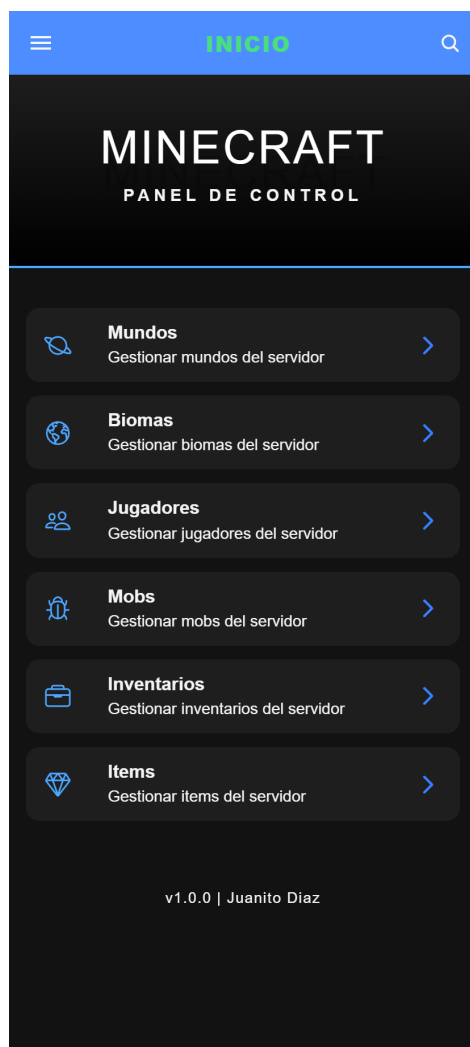


Figura 11 — Panel de bienvenida con accesos directos a todos los módulos.

Módulo de Mundos y Biomas

El módulo de **Mundos** permite visualizar, crear, editar y eliminar los mundos del servidor. Cada mundo muestra su nombre, dificultad y semilla (seed). El módulo de **Biomas** lista las zonas geográficas asociadas a cada mundo con su temperatura registrada en grados Celsius.

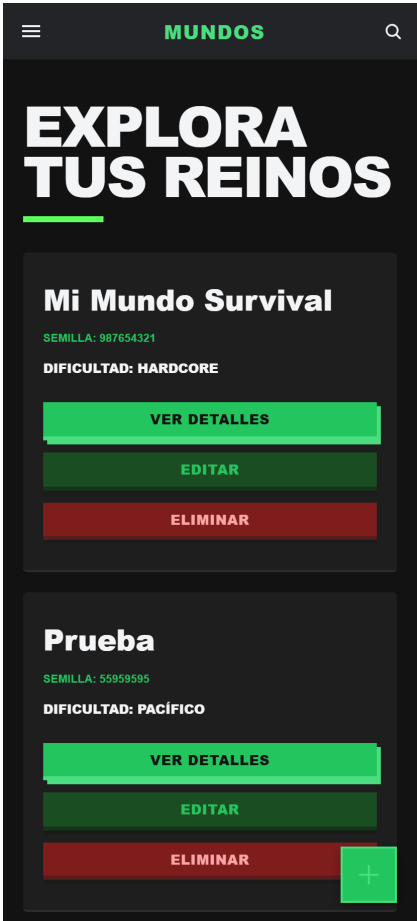


Fig. 12 — Listado de Mundos.

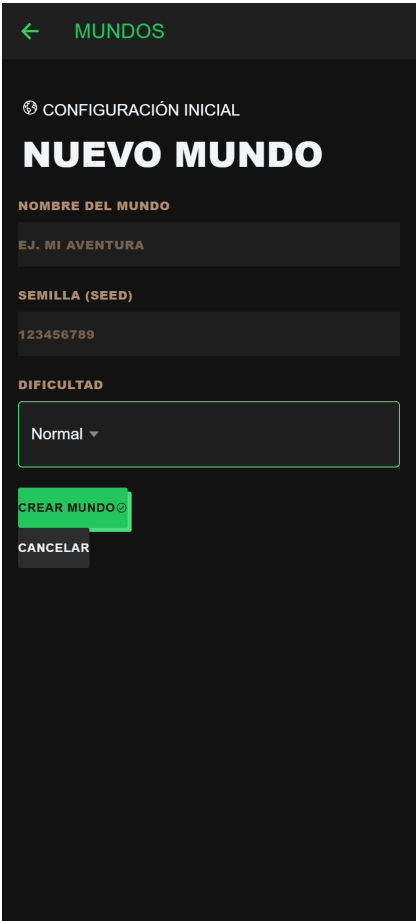


Fig. 13 — Creación de Mundo.

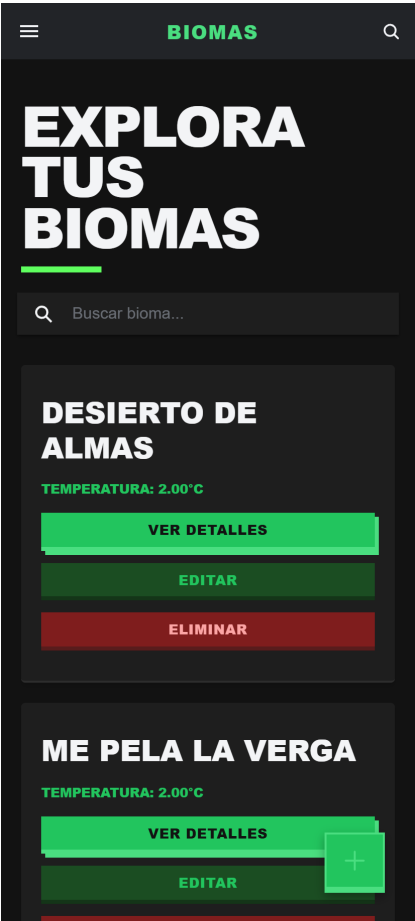


Fig. 14 — Listado de Biomas.

Módulo de Jugadores y Mobs

El módulo de **Jugadores** gestiona los usuarios del servidor incluyendo su nivel de experiencia (XP), rol y foto de perfil. El módulo de **Mobs** administra las criaturas del juego, clasificándolas entre hostiles (como Creepers o Zombies) y pasivas (como vacas o gallinas), y asignándolas a un bioma específico.

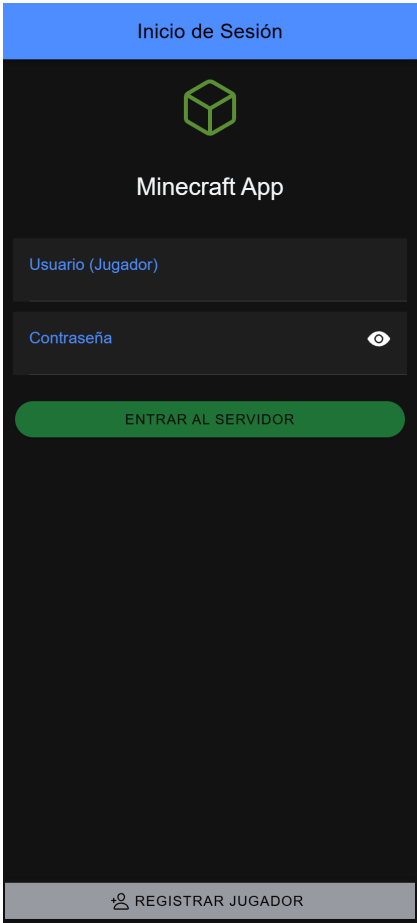


Fig. 15 — Listado de Jugadores.

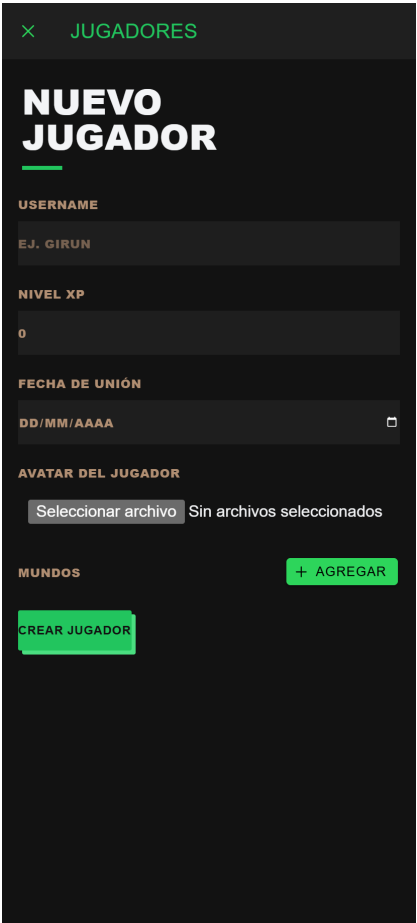


Fig. 16 — Creación de Jugador.

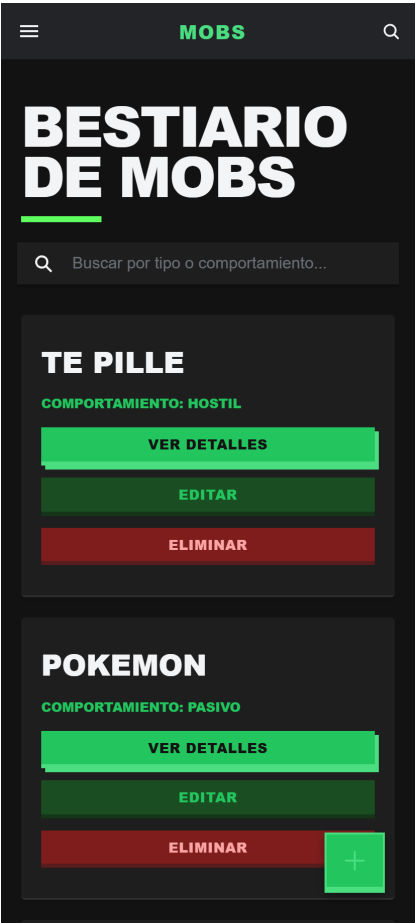


Fig. 17 — Listado de Mobs.

Módulo de Items e Inventarios

El módulo de **Items** registra todos los objetos disponibles en el servidor: herramientas, bloques y recursos, indicando si son apilables y sus puntos de ataque. El módulo de **Inventarios** gestiona la relación entre jugadores e items con la cantidad almacenada, implementando una relación muchos-a-muchos en la base de datos.

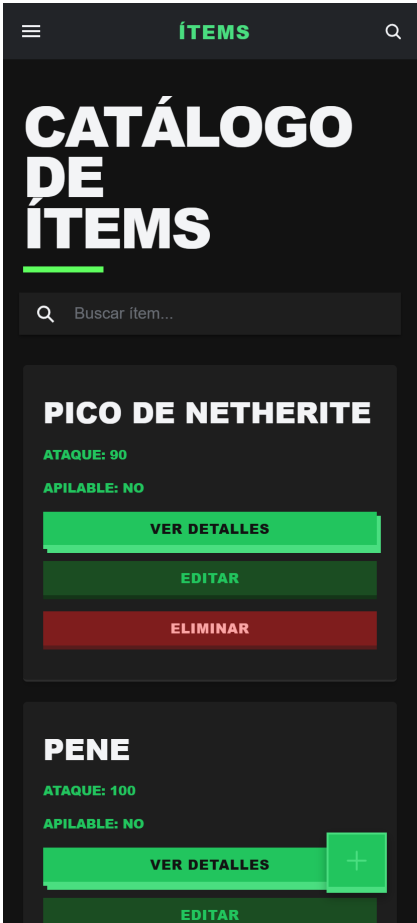


Fig. 18 — Listado de Items.



Fig. 19 — Inventarios por jugador.

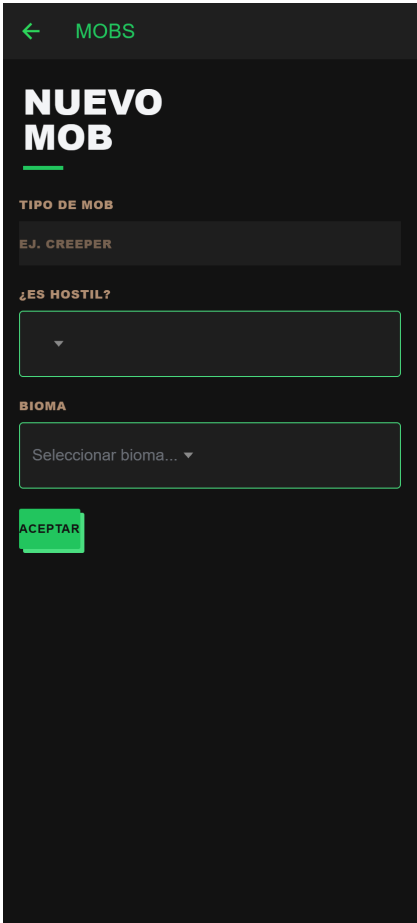


Fig. 20 — Formulario de creación de Mob.

Evidencias del Funcionamiento del Sistema

Flujo Completo de Autenticación

El sistema implementa un flujo de autenticación stateless basado en Bearer Token. Al registrarse o iniciar sesión, el backend genera un `access_token` único que se almacena en `localStorage` del dispositivo. Todas las peticiones a endpoints protegidos incluyen el header `Authorization: Bearer <token>`:

```
Authorization: Bearer Xkj8mP2aQ9nR5sT7vY3wZ6uL4eF1oC0
```

Sistema de Roles y Permisos

La aplicación implementa un sistema de control de acceso basado en roles (RBAC). Cada jugador tiene asignado un rol ('jugador' por defecto o 'administrador'). La tabla **permiso** define qué vistas puede acceder cada rol, siendo consultada inmediatamente después del login para configurar dinámicamente los menús de navegación.

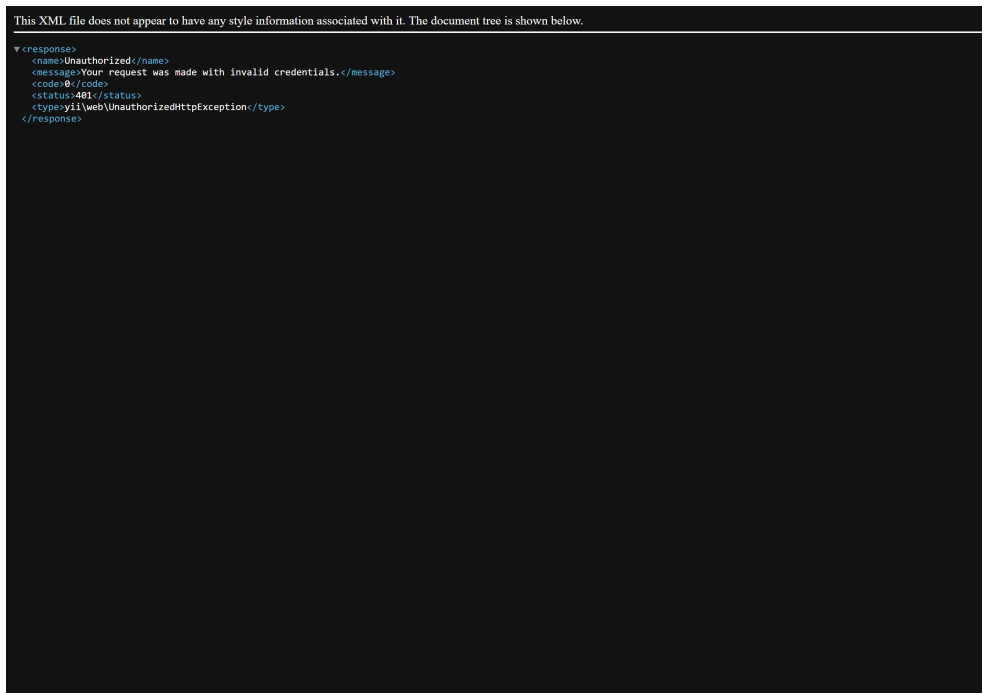


Figura 21 — Endpoint `GET /permiso` retornando permisos por vista y rol.

Relación Muchos a Muchos — Jugadores-Mundos

La tabla pivote **jugadores_mundos** implementa la relación de muchos a muchos entre jugadores y mundos, registrando el timestamp de la última conexión del jugador a cada mundo. Este patrón se implementó usando `viaTable()` en el modelo `ActiveRecord` de Yii 2:

```
return $this->hasMany(Mundos::class, ['id' => 'id_mundo']) ->viaTable('jugadores_mundos',
['id_jugador' => 'id']);
```

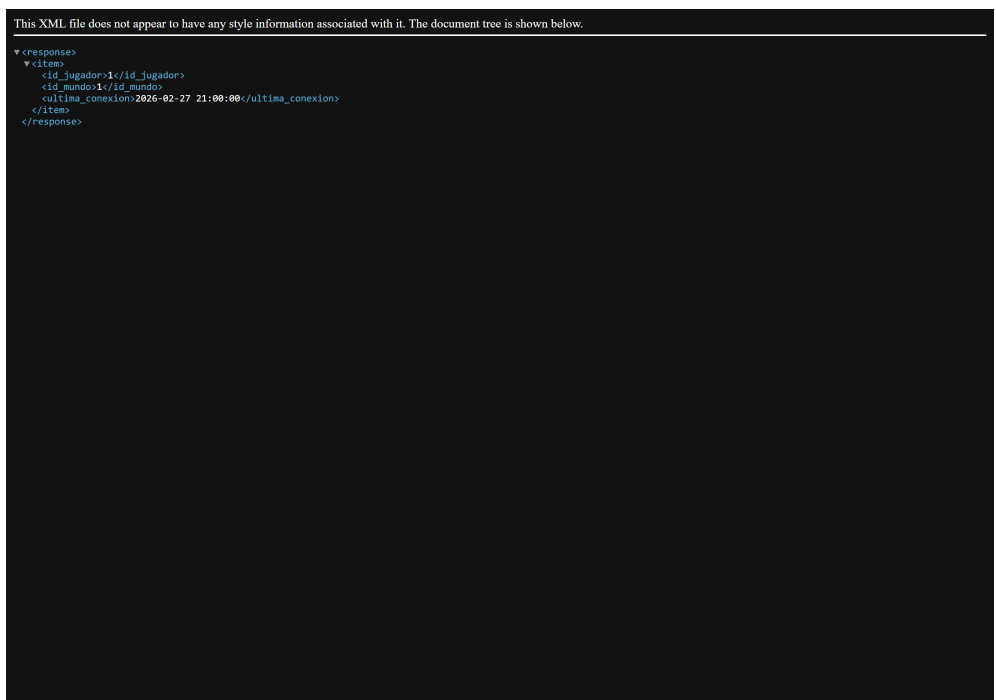


Figura 22 — Endpoint GET /jugadores-mundos mostrando la relación con última conexión.

Conclusión

El desarrollo de la **Aplicación Final U4-T2** ha consolidado de manera integral los conocimientos adquiridos a lo largo del semestre en la materia de Aplicaciones Móviles. La construcción de un sistema full-stack funcional —que combina un backend RESTful en PHP con Yii 2 y un frontend móvil en Ionic/Angular— demostró que el desarrollo de aplicaciones modernas requiere una visión arquitectónica clara y una correcta separación de responsabilidades entre las capas del sistema.

La implementación de la API RESTful con Yii 2 fue uno de los aprendizajes más significativos: comprender cómo el framework genera automáticamente los endpoints CRUD mediante ActiveController, cómo configurar las reglas de autenticación por Bearer Token para proteger selectivamente los endpoints, y cómo gestionar los errores HTTP con los códigos de estado apropiados (200, 201, 401, 422, 500) para que la app móvil pueda reaccionar correctamente a cada situación.

En el lado móvil, la integración de Axios como cliente HTTP dentro de los Observables de RxJS permitió un manejo elegante de las peticiones asíncronas, manteniendo la consistencia con el paradigma reactivo de Angular. El sistema de temas (modo claro/oscuro) implementado con variables CSS de Ionic demostró cómo la experiencia de usuario puede adaptarse dinámicamente al entorno del dispositivo sin duplicar el código de estilos.

Finalmente, la implementación del sistema de roles y permisos mediante una tabla relacional introduce el concepto de RBAC (Role-Based Access Control) de manera práctica, preparando al equipo de desarrollo para sistemas de información reales con múltiples tipos de usuarios y niveles de acceso diferenciados.

Bibliografía

1. Yii Software LLC. (2025). *Yii 2.0 Guide: RESTful Web Services*. Yii Framework Documentation. Recuperado de <https://www.yiiframework.com/doc/guide/2.0/en/rest-quick-start>
2. Ionic Framework. (2026). *Ionic 8 Documentation: Building Cross-Platform Apps*. Ionic Team. Recuperado de <https://ionicframework.com/docs>
3. Angular Team. (2026). *Angular 20 Developer Guide: HTTP Client & Interceptors*. Google. Recuperado de <https://angular.dev/guide/http>
4. Capacitor Team. (2026). *Capacitor 8 — Native Runtime for Web Apps*. Ionic. Recuperado de <https://capacitorjs.com/docs>
5. Axios Contributors. (2025). *Axios: Promise based HTTP client for the browser and Node.js*. Recuperado de <https://axios-http.com/docs/intro>
6. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
7. MySQL AB. (2025). *MySQL 8.0 Reference Manual*. Oracle Corporation. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>